

MODUL PRAKTIKUM

ADMINISTRASI BASIS DATA

SQL Server Management Studio (SSMS)

Database Proyek: D_Monitoring_TA

- Materi 10 — SQL Server Agent & Otomasi Pekerjaan
- Materi 11 — Indeks dan Optimasi Query pada SQL Server
- Materi 13 — Monitoring Performa SQL Server

Universitas Muhammadiyah Yogyakarta

Mata Kuliah: Administrasi Basis Data

Tahun 2025 / 2026

Daftar Isi

Daftar Isi	2
Materi 10 — SQL Server Agent & Otomasi Pekerjaan	3
A. Tujuan	3
B. Alat dan Bahan	3
C. Studi Kasus.....	3
D. Praktikum.....	3
Materi 11 — Indeks dan Optimasi Query pada SQL Server	10
A. Tujuan	10
B. Alat dan Bahan	10
C. Praktikum.....	10
Analisis Hasil Execution Plan — Sebelum Index (Clustered Index Scan).....	14
Analisis Hasil Execution Plan — Sesudah Index (Nonclustered Index Seek)	15
Penjelasan Perubahan.....	17
Materi 13 — Monitoring Performa SQL Server	18
A. Tujuan	18
B. Alat dan Bahan	18
C. Studi Kasus.....	18
D. Praktikum.....	18
Rangkuman Praktikum.....	26

Materi 10 — SQL Server Agent & Otomasi Pekerjaan

A. Tujuan

Setelah mengikuti praktikum ini mahasiswa mampu:

- Mengaktifkan SQL Server Agent pada SQL Server Management Studio (SSMS)
- Membuat Job otomatis untuk backup database D_Monitoring_TA
- Membuat Schedule agar backup berjalan otomatis setiap malam
- Membuat Alerts untuk mendeteksi kegagalan backup secara otomatis
- Membuat Operator sebagai penerima notifikasi email ketika job gagal
- Mengotomasi maintenance database melalui Rebuild Index terjadwal

B. Alat dan Bahan

Software:

- Microsoft SQL Server
- SQL Server Management Studio (SSMS)

Database:

Database yang digunakan pada praktikum ini adalah D_Monitoring_TA — database sistem monitoring tugas akhir mahasiswa. Tabel utama yang digunakan adalah aktivitas.Log_Bimbingan yang menyimpan seluruh riwayat bimbingan antara mahasiswa dan dosen pembimbing.

C. Studi Kasus

Konteks Sistem

Database D_Monitoring_TA digunakan untuk mencatat aktivitas bimbingan skripsi mahasiswa secara harian.

Tabel aktivitas.Log_Bimbingan akan terus bertambah isinya seiring berjalannya waktu.

Administrator database ingin memastikan:

- Data aman dengan backup otomatis setiap malam
- Performa tabel terjaga dengan rebuild index rutin setiap minggu
- Administrator mendapat notifikasi email jika backup gagal

D. Praktikum

PRAKTIKUM 1 — Mengaktifkan SQL Server Agent

Langkah 1 Buka SQL Server Management Studio (SSMS) dan connect ke server.

Langkah 2 Di Object Explorer, temukan node SQL Server Agent.

Node SQL Server Agent berada di bawah nama server, sejajar dengan Databases, Security, dan Server Objects.

Langkah 3 Klik kanan SQL Server Agent → pilih Start.

Langkah 4 Verifikasi status Agent sudah Running.

Jika berhasil, ikon di samping SQL Server Agent akan berubah menjadi hijau dan teks menunjukkan SQL Server Agent (Running). Setelah Running, node Jobs, Alerts, Operators, dan Proxies akan dapat diakses.

Catatan

Jika SQL Server Agent tidak bisa di-Start, kemungkinan penyebabnya:

1. Layanan SQL Server Agent dinonaktifkan di Windows Services
2. Akun Windows tidak memiliki hak yang cukup

Solusi: buka Services (services.msc), cari SQL Server Agent, set Startup Type ke Automatic.

PRAKTIKUM 2 — Membuat Job Backup Otomatis

Langkah 1 Buat Job baru.

Klik kanan Jobs di bawah SQL Server Agent → pilih New Job.

Langkah 2 Isi nama Job pada tab General.

Field	Nilai yang Diisi
Name	Backup_Harian_MonitoringTA
Owner	sa (atau akun administrator)
Description	Job backup harian database D_Monitoring_TA secara otomatis

Langkah 3 Buat Job Step — masuk tab Steps → klik New.

Field	Nilai yang Diisi
Step name	Backup Database
Type	Transact-SQL script (T-SQL)
Database	D_Monitoring_TA
Command	(lihat query di bawah)

Langkah 4 Masukkan query backup berikut pada kolom Command.

```
BACKUP DATABASE [D_Monitoring_TA]
TO DISK = 'D:\BackupSQL\MonitoringTA_Full.bak'
WITH FORMAT,
    MEDIANAME = 'BackupDB',
    NAME = 'Full Backup MonitoringTA';
GO
```

Perintah T-SQL	Fungsi
BACKUP DATABASE [D_Monitoring_TA]	Mendefinisikan database yang akan dibackup
TO DISK = '...'	Menentukan lokasi dan nama file backup (.bak)
WITH FORMAT	Membuat file backup baru (menimpa file lama jika ada)
MEDIANAME = 'BackupDB'	Memberi nama media backup
NAME = 'Full Backup MonitoringTA'	Memberi nama deskriptif pada backup set ini

Pastikan Folder Tersedia

Sebelum menjalankan job, pastikan folder D:\BackupSQL\ sudah ada di komputer.

Jika belum ada, buat manual di Windows Explorer atau gunakan perintah:

```
EXEC xp_cmdshell 'mkdir D:\BackupSQL'
```

Atau ubah path sesuai folder yang tersedia, misalnya C:\BackupSQL\

Langkah 5 Klik OK untuk menyimpan Job Step.

PRAKTIKUM 3 — Membuat Schedule Backup Otomatis

Masih di jendela New Job yang sama, masuk ke tab Schedules → klik New.

Langkah 1 Isi konfigurasi Schedule sebagai berikut.

Field	Nilai
Name	Backup_Malam
Schedule type	Recurring
Occurs	Daily (setiap hari)
Daily frequency	Occurs once at: 23:00:00
Start date	Tanggal hari ini
End date	No end date (berjalan terus)

Langkah 2 Klik OK untuk menyimpan Schedule, lalu klik OK lagi untuk menyimpan Job.

Hasil: Job Backup_Harian_MonitoringTA akan berjalan otomatis setiap malam pukul 23:00, membuat file backup D_Monitoring_TA di folder D:\BackupSQL\.

Cara Menjalankan Job Secara Manual (untuk Test)

Klik kanan job Backup_Harian_MonitoringTA → pilih Start Job at Step...

Jika berhasil, akan muncul pesan 'Job succeeded'.

Cek folder D:\BackupSQL\ — file MonitoringTA_Full.bak harus sudah ada.

PRAKTIKUM 4 — Membuat Maintenance Job (Rebuild Index)

Studi Kasus

Tabel aktivitas.Log_Bimbingan akan sering menerima INSERT data bimbingan baru.

Seiring waktu, indeks pada tabel ini akan mengalami fragmentasi sehingga query menjadi lambat.

Solusi: buat job otomatis untuk melakukan Rebuild Index setiap minggu.

Langkah 1 Buat Job baru: SQL Server Agent → Jobs → klik kanan → New Job.

Field	Nilai
Name	Rebuild_Index_MonitoringTA
Description	Rebuild semua indeks database D_Monitoring_TA setiap minggu

Langkah 2 Buat Job Step baru pada tab Steps → New.

Field	Nilai
Step name	Rebuild All Index

Field	Nilai
Type	Transact-SQL script (T-SQL)
Database	D_Monitoring_TA

Langkah 3 Masukkan query berikut pada kolom Command.

```
USE D_Monitoring_TA;
GO
EXEC sp_MSforeachtable
    'ALTER INDEX ALL ON ? REBUILD';
GO
```

Perintah	Fungsi
sp_MSforeachtable	Menjalankan perintah ke semua tabel dalam database secara otomatis
ALTER INDEX ALL ON ? REBUILD	Melakukan rebuild pada semua indeks di setiap tabel (? = nama tabel)

Langkah 4 Buat Schedule untuk Job ini pada tab Schedules → New.

Field	Nilai
Name	Weekly_Index_Rebuild
Schedule type	Recurring
Occurs	Weekly — pilih hari Minggu
Daily frequency	Occurs once at: 01:00:00

Hasil: setiap Minggu dini hari pukul 01:00, SQL Server akan secara otomatis melakukan rebuild pada seluruh indeks database D_Monitoring_TA.

PRAKTIKUM 5 — Membuat Operator (Penerima Notifikasi)

Operator adalah entitas dalam SQL Server Agent yang mewakili orang atau tim yang akan menerima notifikasi. Notifikasi biasanya dikirim melalui email ketika sebuah job gagal atau alert terpicu.

Langkah 1 Masuk ke SQL Server Agent → klik kanan Operators → New Operator.

Langkah 2 Isi data Operator sebagai berikut.

Field	Nilai
Name	Admin_MonitoringTA
E-mail name	rrassya19@gmail.com
Enabled	Centang (aktifkan)

Langkah 3 Klik OK untuk menyimpan Operator.

Catatan Database Mail

Agar email dapat terkirim, SQL Server harus dikonfigurasi dengan Database Mail terlebih dahulu. Konfigurasi Database Mail: Management → Database Mail → Configure Database Mail. Tanpa Database Mail, Operator tetap bisa dibuat namun notifikasi email tidak akan terkirim.

PRAKTIKUM 6 — Membuat Alert Kegagalan Backup

Studi Kasus

Jika job backup di Praktikum 2 gagal (misalnya folder tujuan penuh atau tidak ada), sistem harus secara otomatis mengirim email notifikasi ke Admin_MonitoringTA. Ini dilakukan dengan membuat Alert yang terpicu pada error SQL Server nomor 3013.

Langkah 1 Masuk ke SQL Server Agent → klik kanan Alerts → New Alert.

Langkah 2 Isi konfigurasi pada tab General.

Field	Nilai
Name	Alert_Backup_Gagal
Type	SQL Server event alert
Database name	D_Monitoring_TA
Error number	3013
Severity	(kosongkan — gunakan error number)

Error 3013

Error number 3013 adalah kode error SQL Server yang berarti:
'BACKUP DATABASE is terminating abnormally.'
Error ini muncul ketika backup database gagal karena berbagai sebab:

- Folder tujuan tidak ada atau penuh
- Tidak ada izin menulis ke folder tujuan
- Disk penuh

Langkah 3 Atur Respons Alert pada tab Response.

- Centang opsi Notify operators
- Pada kolom Admin_MonitoringTA, centang kolom E-mail
- Klik OK untuk menyimpan Alert

Langkah 4 Test Alert dengan mensimulasikan kegagalan backup.

Cara test: ubah sementara path backup pada Job Step menjadi folder yang tidak ada, misalnya:

```
BACKUP DATABASE [D_Monitoring_TA]
TO DISK = 'Z:\FolderTidakAda\test.bak' -- folder tidak ada
WITH FORMAT, NAME = 'Test Backup Gagal';
```

Jalankan job tersebut secara manual. SQL Server akan menghasilkan error 3013, Alert terpicu, dan email notifikasi dikirim ke Admin_MonitoringTA. Kembalikan path ke yang benar setelah selesai testing.

Materi 11 — Indeks dan Optimasi Query pada SQL Server

A. Tujuan

Setelah mengikuti praktikum ini mahasiswa mampu:

- Mengamati performa query pada tabel aktivitas.Log_Bimbingan tanpa indeks
- Mengidentifikasi Clustered Index yang sudah ada pada tabel tersebut
- Membuat Non-Clustered Index pada kolom Nim dan kolom Tanggal
- Membandingkan performa query sebelum dan sesudah pengindeksan
- Menggunakan SET STATISTICS TIME dan SET STATISTICS IO untuk mengukur performa
- Menganalisis Execution Plan untuk memahami cara SQL Server mengeksekusi query

B. Alat dan Bahan

Software:

- Microsoft SQL Server
- SQL Server Management Studio (SSMS)

Database:

Database D_Monitoring_TA, tabel aktivitas.Log_Bimbingan.

Kolom	Tipe Data	Keterangan
ID_Log	INT (IDENTITY)	Primary Key — Clustered Index otomatis dibuat di sini
Nim	VARCHAR	NIM mahasiswa — sering digunakan sebagai filter pencarian
NIDN	VARCHAR	NIDN dosen pembimbing
Tanggal	DATE	Tanggal bimbingan berlangsung
Materi	NVARCHAR(MAX)	Isi/topik yang dibahas saat bimbingan
Status	VARCHAR	Status bimbingan: Selesai, Proses, Revisi, Disetujui

C. Praktikum

PRAKTIKUM 1 — Mengamati Query Tanpa Indeks

Sebelum membuat indeks apapun, kita perlu mengukur performa awal query pencarian pada tabel aktivitas.Log_Bimbingan. Ini menjadi baseline perbandingan.

Langkah 1 Aktifkan statistik performa.

```
USE D_Monitoring_TA;
GO
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO
```

Perintah	Fungsi	Output yang Ditampilkan
SET STATISTICS TIME ON	Mengukur waktu eksekusi query	CPU time dan Elapsed time (dalam ms)
SET STATISTICS IO ON	Mengukur aktivitas baca/tulis ke disk dan memori	Logical reads, physical reads, dan scan count

Langkah 2 Jalankan query pencarian berdasarkan Nim.

```
SELECT *
FROM aktivitas.Log_Bimbingan
WHERE Nim = '20250140157';
GO
```

Langkah 3 Catat hasil statistik dari tab Messages.

Metrik	Nilai yang Dicatat	Interpretasi
Scan count	...	Berapa kali SQL Server melakukan scan/seek pada indeks
Logical reads	... pages	Jumlah halaman dibaca dari cache (RAM). 1 page = 8 KB
Physical reads	... pages	Jumlah halaman dibaca dari disk. Idealnya 0 (sudah di cache)
CPU time	... ms	Waktu prosesor yang digunakan
Elapsed time	... ms	Total waktu eksekusi dari awal hingga selesai

Interpretasi Logical Reads

Logical Reads Interpretasi

1 – 10 : Sangat Baik

10 – 100 : Baik

100 – 1000 : Mulai Berat, pertimbangkan optimasi

> 1000 : Perlu Optimasi segera

Physical Reads = 0 artinya data sudah ada di Buffer Cache (RAM) — sangat baik.
Physical Reads > 0 artinya SQL Server harus membaca dari disk — lebih lambat.

PRAKTIKUM 2 — Mengidentifikasi Clustered Index yang Ada

Sebelum membuat indeks baru, periksa indeks yang sudah ada pada tabel aktivitas.Log_Bimbingan menggunakan stored procedure sp_helpIndex.

Langkah 1 Jalankan perintah berikut.

```
USE D_Monitoring_TA;
GO
EXEC sp_helpIndex 'aktivitas.Log_Bimbingan';
GO
```

Langkah 2 Amati hasil dan isi tabel berikut.

Kolom Hasil	Nilai yang Ditemukan	Penjelasan
index_name	PK_Log_Bimbingan	Nama indeks Primary Key yang otomatis dibuat SQL Server
index_description	clustered, unique, primary key...	Jenis indeks: Clustered = data disimpan berurutan sesuai indeks ini
index_keys	ID_Log	Kolom yang menjadi kunci indeks (Primary Key)

Kesimpulan: tabel aktivitas.Log_Bimbingan sudah memiliki Clustered Index pada kolom ID_Log (Primary Key). Kolom Nim dan Tanggal belum memiliki indeks, sehingga pencarian berdasarkan dua kolom tersebut menggunakan Table Scan.

PRAKTIKUM 3 — Membuat Non-Clustered Index pada Kolom Nim

Studi Kasus

Bagian akademik sering mencari log bimbingan berdasarkan NIM mahasiswa untuk memantau perkembangan bimbingan setiap mahasiswa.

Query: `SELECT * FROM aktivitas.Log_Bimbingan WHERE Nim = '...'`

Tanpa indeks, SQL Server harus memindai seluruh tabel (Clustered Index Scan).

Solusi: buat Non-Clustered Index pada kolom Nim.

Langkah 1 Catat performa SEBELUM indeks dibuat (sudah dilakukan di Praktikum 1).

Kondisi	Logical Reads	CPU Time	Elapsed Time	Operasi SQL Server
Sebelum Index	...	... ms	... ms	Clustered Index Scan
Sesudah Index	...	... ms	... ms	(isi setelah Langkah 3)

Langkah 2 Buat Non-Clustered Index pada kolom Nim.

```
USE D_Monitoring_TA;
GO
CREATE NONCLUSTERED INDEX IDX_Log_Bimbingan_Nim
ON aktivitas.Log_Bimbingan (Nim);
GO
```

Langkah 3 Jalankan kembali query pencarian dan catat performa SESUDAH indeks.

```
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO
SELECT *
FROM aktivitas.Log_Bimbingan
WHERE Nim = '20250140157';
GO
```

Isi kolom 'Sesudah Index' pada tabel di Langkah 1. Logical Reads seharusnya berkurang dan operasi berubah menjadi Non-Clustered Index Seek.

PRAKTIKUM 4 — Analisis Execution Plan

Execution Plan adalah representasi visual dari langkah-langkah yang diambil SQL Server untuk mengeksekusi query. Ini adalah alat terpenting untuk memahami dan mengoptimasi performa query.

Langkah 1 Aktifkan Actual Execution Plan.

Tekan Ctrl + M di SSMS (atau klik ikon 'Include Actual Execution Plan' pada toolbar).

Langkah 2 Jalankan query berikut.

```
USE D_Monitoring_TA;
GO
SELECT *
FROM aktivitas.Log_Bimbingan
WHERE Nim = '20250140157';
GO
```

Langkah 3 Klik tab Execution Plan dan amati detailnya.

Klik ikon operator pada Execution Plan untuk melihat properti detail. Isi tabel analisis berikut:

Properti Execution Plan	Nilai Sebelum Index	Nilai Sesudah Index	Penjelasan
Physical Operation	Clustered Index Scan	Nonclustered Index Seek	Scan = baca semua, Seek = langsung ke data
Logical Operation	Clustered Index Scan	Nonclustered Index Seek	Harus sama dengan Physical Operation
Actual Rows Read	...	1	Jumlah baris yang benar-benar dibaca SQL Server
Actual Rows Returned	...	1	Jumlah baris yang dikembalikan ke pengguna
Estimated Operator Cost	...	...	Perkiraan biaya eksekusi (semakin kecil semakin baik)
Ordered	False	True	True = data dibaca dalam urutan indeks (lebih efisien)
Seek Predicate / Predicate	Predicate: Nim = '...'	Seek Predicate: Nim = '...'	Seek Predicate lebih efisien dari Predicate biasa

Analisis Hasil Execution Plan — Sebelum Index (Clustered Index Scan)

Physical & Logical Operation = Clustered Index Scan

SQL Server memindai seluruh isi tabel melalui Clustered Index PK_Log_Bimbingan karena kolom Nim belum memiliki indeks tersendiri. Seluruh baris dibaca satu per satu hingga menemukan Nim yang cocok.

Ordered = False

Data tidak dibaca dalam urutan tertentu. SQL Server tidak dapat memanfaatkan struktur B-Tree untuk langsung menuju lokasi data, sehingga harus membaca dari awal hingga akhir tabel.

Predicate (bukan Seek Predicate)

Karena SQL Server melakukan scan, kondisi WHERE Nim = '20250140157' diterapkan sebagai filter setelah data dibaca, bukan sebagai panduan pencarian. Ini jauh kurang efisien dibanding Seek Predicate.

Estimated Row Size = 4075 B

Ukuran baris sangat besar karena mencakup semua kolom termasuk kolom Materi (NVARCHAR MAX). Ini menambah beban I/O saat scan.

Analisis Hasil Execution Plan — Sesudah Index (Nonclustered Index Seek)

Physical & Logical Operation = Nonclustered Index Seek

SQL Server kini menggunakan Non-Clustered Index IDX_Log_Bimbingan_Nim untuk langsung menavigasi ke lokasi data melalui struktur B-Tree. Tidak ada pemindaian seluruh tabel.

Ordered = True

Data dibaca dalam urutan indeks Nim, menunjukkan SQL Server memanfaatkan struktur B-Tree secara penuh. Proses lebih efisien karena navigasi langsung ke node yang tepat.

Seek Predicate = Nim = '20250140157'

Kondisi WHERE digunakan sebagai Seek Predicate, artinya kondisi ini digunakan langsung sebagai panduan navigasi dalam B-Tree, bukan sekadar filter setelah data dibaca.

Kesimpulan Analisis

Dengan membuat Non-Clustered Index IDX_Log_Bimbingan_Nim, SQL Server beralih dari Clustered Index Scan (membaca seluruh tabel) menjadi Nonclustered Index Seek (langsung ke data). Logical Reads berkurang drastis, dan waktu eksekusi lebih cepat. Semakin banyak data dalam tabel Log_Bimbingan, semakin besar manfaat indeks ini terasa.

PRAKTIKUM 5 — Optimasi Query ORDER BY dengan Index pada Kolom Tanggal

Studi Kasus

Administrator ingin menampilkan seluruh log bimbingan yang diurutkan berdasarkan tanggal untuk melihat kronologi bimbingan mahasiswa.

Query: `SELECT * FROM aktivitas.Log_Bimbingan ORDER BY Tanggal;`

Tanpa indeks pada kolom Tanggal, SQL Server harus membaca semua data lalu melakukan operasi Sort terpisah — dua tahap yang mahal secara performa.

Langkah 1 Aktifkan Execution Plan (Ctrl+M) lalu jalankan query SEBELUM index dibuat.

```
USE D_Monitoring_TA;
GO
SELECT *
FROM aktivitas.Log_Bimbingan
ORDER BY Tanggal;
GO
```

Amati Execution Plan — perhatikan ada dua node operator: Clustered Index Scan dan Sort. Catat Cost % masing-masing node.

Langkah 2 Buat Non-Clustered Index pada kolom Tanggal.

```
USE D_Monitoring_TA;
GO
CREATE NONCLUSTERED INDEX IDX_Log_Bimbingan_Tanggal
ON aktivitas.Log_Bimbingan (Tanggal);
GO
```

Langkah 3 Jalankan kembali query ORDER BY dan amati perubahan Execution Plan.

```
SELECT *
FROM aktivitas.Log_Bimbingan
ORDER BY Tanggal;
GO
```

Langkah 4 Bandingkan hasilnya dan isi tabel perbandingan berikut.

Aspek	Sebelum Index Tanggal	Sesudah Index Tanggal
Physical Operation	Clustered Index Scan	Clustered Index Scan (Node 2)
Estimated Operator Cost	... (22%)	... (13%)
Estimated Row Size	4075 B	14 B
Output List	Semua kolom (ID_Log, Nim, NIDN, Tanggal, Materi, Status)	ID_Log dan Tanggal saja
Node ID	1	2
Ordered	False	False
Operasi Sort	Ada (Cost ~78%)	Masih ada, namun lebih ringan

Penjelasan Perubahan

Mengapa Estimated Row Size turun dari 4075 B menjadi 14 B?

Sesudah index Tanggal dibuat, SQL Server memanfaatkan Non-Clustered Index untuk membaca hanya kolom ID_Log dan Tanggal terlebih dahulu (14 B per baris). Setelah mendapatkan urutan Tanggal, baru dilakukan Key Lookup ke tabel utama untuk mengambil kolom lainnya. Ini jauh lebih efisien dari membaca semua kolom sekaligus di setiap baris.

Mengapa Physical Operation masih Clustered Index Scan?

Karena query menggunakan SELECT * (semua kolom), SQL Server tetap harus mengakses tabel utama melalui Clustered Index untuk mendapatkan kolom Materi, Nim, NIDN, dan Status. Non-Clustered Index Tanggal hanya membantu SQL Server menentukan urutan baris, bukan menghilangkan kebutuhan akses ke tabel utama.

Kesimpulan

Non-Clustered Index IDX_Log_Bimbingan_Tanggal berhasil mengurangi Estimated Operator Cost dari 22% menjadi 13% dan menurunkan Estimated Row Size dari 4075 B menjadi 14 B. SQL Server kini hanya membaca kolom yang diperlukan dari indeks terlebih dahulu, sehingga beban I/O berkurang signifikan. Manfaat ini akan semakin besar seiring bertambahnya jumlah data dalam tabel Log_Bimbingan.

Langkah 5 Cleanup — hapus indeks setelah selesai praktikum (opsional).

```
-- Hapus index setelah praktikum (opsional, untuk menjaga database bersih)
DROP INDEX IDX_Log_Bimbingan_Nim ON aktivitas.Log_Bimbingan;
DROP INDEX IDX_Log_Bimbingan_Tanggal ON aktivitas.Log_Bimbingan;
GO
```

Materi 13 — Monitoring Performa SQL Server

A. Tujuan

Setelah mengikuti praktikum ini mahasiswa mampu:

- Menggunakan Activity Monitor untuk memantau aktivitas SQL Server secara real-time
- Mengidentifikasi query pada database D_Monitoring_TA yang menggunakan resource tinggi
- Menggunakan Performance Monitor (PerfMon) untuk memantau CPU, memori, dan disk
- Menggunakan SQL Server Profiler untuk merekam aktivitas query
- Mengidentifikasi query lambat (slow query) menggunakan filter Duration
- Memahami konsep blocking dan deadlock dalam konteks database multi-user

B. Alat dan Bahan

Software:

- Microsoft SQL Server
- SQL Server Management Studio (SSMS)
- SQL Server Profiler
- Windows Performance Monitor (PerfMon)

Database:

Database D_Monitoring_TA, tabel aktivitas.Log_Bimbingan.

C. Studi Kasus

Konteks

Database D_Monitoring_TA digunakan oleh banyak pengguna: mahasiswa, dosen, dan admin akademik.

Seiring bertambahnya data bimbingan, administrator perlu memantau performa database.

Target monitoring:

- Mengetahui kondisi server secara real-time
- Menemukan query pada tabel Log_Bimbingan yang paling banyak menggunakan resource
- Mendeteksi query yang berjalan lambat
- Memantau penggunaan CPU, memori, dan disk server

D. Praktikum

PRAKTIKUM 1 — Monitoring dengan Activity Monitor

Activity Monitor adalah alat bawaan SSMS yang menampilkan kondisi server secara real-time. Ini adalah alat pertama yang dibuka DBA ketika server terasa lambat.

Langkah 1 Buka Activity Monitor.

Pilih salah satu cara berikut:

- Object Explorer → klik kanan nama server → Activity Monitor
- Atau tekan Ctrl + Alt + A

Langkah 2 Amati panel Overview.

Panel Overview	Informasi yang Ditampilkan	Kondisi Normal
% Processor Time (CPU Usage)	Persentase penggunaan prosesor oleh SQL Server	< 70% rata-rata
Waiting Tasks	Jumlah tugas yang menunggu sumber daya (lock, I/O, dll.)	Mendekati 0
Database I/O (MB/sec)	Kecepatan baca/tulis data ke storage	Tergantung workload
Batch Requests/sec	Jumlah query yang diproses per detik	Stabil sesuai beban normal

Langkah 3 Jalankan query berikut di SSMS lalu amati perubahan pada Activity Monitor.

```
USE D_Monitoring_TA;
GO
SELECT * FROM aktivitas.Log_Bimbingan;
GO
```

Langkah 4 Catat hasil pengamatan pada tabel berikut.

Parameter	Nilai Sebelum Query	Nilai Saat Query Berjalan	Keterangan
CPU Usage (%)			
Waiting Tasks			
Database I/O (MB/sec)			

Parameter	Nilai Sebelum Query	Nilai Saat Query Berjalan	Keterangan
Batch Requests/sec			

PRAKTIKUM 2 — Melihat Recent Expensive Queries

Panel Recent Expensive Queries pada Activity Monitor menampilkan query-query yang paling banyak menggunakan sumber daya (CPU, memori, dan I/O) dalam periode waktu tertentu.

Langkah 1 Pada Activity Monitor, expand panel Recent Expensive Queries.

Langkah 2 Jalankan query berikut beberapa kali berulang di SSMS (klik Execute 3–5 kali).

```
USE D_Monitoring_TA;
GO
SELECT * FROM aktivitas.Log_Bimbingan
ORDER BY Tanggal;
GO
```

Langkah 3 Perhatikan query tersebut muncul di panel Recent Expensive Queries, lalu catat hasilnya.

Query	CPU	Duration (ms)	Logical Reads	Database
SELECT * FROM aktivitas.Log_Bimbingan ORDER BY Tanggal				D_Monitoring_TA

Kolom-Kolom di Recent Expensive Queries

CPU : Total CPU time yang digunakan query (dalam ms)

Duration : Total waktu eksekusi dari awal hingga selesai (dalam ms)

Logical Reads : Jumlah halaman data yang dibaca dari cache memori

Physical Reads: Jumlah halaman yang dibaca langsung dari disk (idealnya 0)

Database : Nama database tempat query dijalankan

Query dengan CPU dan Duration tinggi adalah kandidat utama untuk dioptimasi.

PRAKTIKUM 3 — Monitoring CPU dan Memory dengan PerfMon

Performance Monitor (PerfMon) adalah alat Windows yang dapat memantau seluruh sistem server, bukan hanya SQL Server. Ini berguna untuk memastikan apakah masalah performa berasal dari SQL Server atau dari keterbatasan hardware.

Langkah 1 Buka Performance Monitor.

- Tekan Windows + R
- Ketik perfmon lalu tekan Enter

Langkah 2 Masuk ke Performance Monitor dan tambahkan Counter.

Klik Performance Monitor di panel kiri, lalu klik tombol + (Add Counters) di toolbar.

Langkah 3 Tambahkan counter berikut satu per satu.

Counter	Kategori	Instance	Threshold Perhatian
% Processor Time	Processor	_Total	> 80% secara terus-menerus
Available MBytes	Memory	(tidak ada)	< 10% dari total RAM
Avg. Disk Queue Length	PhysicalDisk	_Total	> 2

Langkah 4 Buat beban server untuk melihat grafik PerfMon bergerak.

Pertama, insert data dummy ke tabel Log_Bimbingan agar ada data yang cukup:

```
-- Cek apakah ID_Log adalah IDENTITY
SELECT is_identity
FROM sys.columns
WHERE object_id = OBJECT_ID('aktivitas.Log_Bimbingan') AND name = 'ID_Log';
```

```
-- Insert 5000 baris data dummy untuk pengujian
INSERT INTO aktivitas.Log_Bimbingan (Nim, NIDN, Tanggal, Materi, Status)
SELECT
    (SELECT TOP 1 Nim FROM master.Mahasiswa ORDER BY NEWID()),
    (SELECT TOP 1 NIDN FROM master.Dosen ORDER BY NEWID()),
    CAST DATEADD(DAY, ABS(CHECKSUM(NEWID())) % 1260, '2023-01-01') AS DATE,
    (SELECT TOP 1 Materi FROM (VALUES
        (N'Konsultasi Bab 1 - Pendahuluan'),
        (N'Revisi rumusan masalah penelitian'),
        (N'Pembahasan metodologi penelitian'),
        (N'Konsultasi Bab 2 - Tinjauan Pustaka'),
```

```

        (N'Review instrumen penelitian'),
        (N'Pembahasan hasil analisis data'),
        (N'Revisi Bab 4 - Hasil dan Pembahasan'),
        (N'Konsultasi kesimpulan dan saran'),
        (N'Persiapan sidang pendadaran'),
        (N'Revisi format penulisan skripsi')
    ) AS T(Materi) ORDER BY NEWID()),
    (SELECT TOP 1 Status FROM (VALUES
        ('Selesai'), ('Proses'), ('Revisi'), ('Disetujui')
    ) AS S(Status) ORDER BY NEWID())
FROM (
    SELECT TOP (5000) ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS n
    FROM sys.all_objects a CROSS JOIN sys.all_objects b
) AS Angka;

-- Cek jumlah data setelah insert
SELECT COUNT(*) AS Total_Log_Bimbingan FROM aktivitas.Log_Bimbingan;

```

Kemudian jalankan query berikut berulang kali untuk membuat beban ke server:

```

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
SELECT * FROM aktivitas.Log_Bimbingan;
GO 50 -- jalankan 50 kali berturut-turut

```

Langkah 5 Amati perubahan grafik pada PerfMon dan catat hasilnya.

Counter	Nilai Awal (Idle)	Nilai Saat Query Berjalan	Keterangan
% Processor Time			
Available MBytes			
Avg. Disk Queue Length			

Langkah 6 Bersihkan data dummy setelah selesai.

```

-- Cek data asli yang ada sebelum insert dummy
SELECT TOP 5 * FROM aktivitas.Log_Bimbingan ORDER BY ID_Log ASC;

-- Hapus data dummy (ganti angka 1 sesuai ID_Log data asli terakhir)
DELETE FROM aktivitas.Log_Bimbingan
WHERE ID_Log > 1; -- sesuaikan dengan ID_Log data asli kamu

```

```
-- Verifikasi jumlah data setelah hapus
SELECT COUNT(*) AS Total_Log_Bimbingan FROM aktivitas.Log_Bimbingan;
```

PRAKTIKUM 4 — Merekam Aktivitas Query dengan SQL Server Profiler

SQL Server Profiler memungkinkan DBA merekam semua aktivitas yang terjadi pada SQL Server secara detail: query yang dieksekusi, siapa yang menjalankannya, kapan, berapa lama, dan berapa resource yang digunakan.

Langkah 1 Buka SQL Server Profiler.

Di SSMS: menu Tools → SQL Server Profiler.

Langkah 2 Buat trace baru.

File → New Trace → koneksikan ke server.

Langkah 3 Pilih Template: Standard.

Langkah 4 Pada tab Events Selection, pastikan event berikut dicentang.

Event	Kategori	Kegunaan
SQL:BatchCompleted	TSQL	Merekam query batch yang selesai dieksekusi beserta statistiknya
RPC:Completed	Stored Procedures	Merekam stored procedure yang selesai dieksekusi

Langkah 5 Klik Run untuk mulai merekam.

Langkah 6 Kembali ke SSMS dan jalankan query berikut.

```
USE D_Monitoring_TA;
GO
SELECT *
FROM aktivitas.Log_Bimbingan
WHERE Status = 'Disetujui';
GO
```

Langkah 7 Amati hasil rekaman di Profiler dan catat.

Kolom Profiler	Nilai yang Terekam	Keterangan
TextData	SELECT * FROM aktivitas.Log_Bimbingan WHERE Status = 'Disetujui'	Teks query yang dieksekusi
StartTime		Waktu mulai eksekusi query
EndTime		Waktu selesai eksekusi query
Duration		Lama eksekusi dalam milliseconds
CPU		CPU time dalam milliseconds
Reads		Jumlah logical reads
NTUserName / LoginName		Akun yang menjalankan query

PRAKTIKUM 5 — Mencari Query yang Lambat (Slow Query)

Query lambat adalah query yang membutuhkan waktu lebih dari yang seharusnya. SQL Server Profiler dapat dikonfigurasi untuk hanya menampilkan query yang melampaui batas durasi tertentu.

Langkah 1 Pada SQL Server Profiler yang masih berjalan, buka Column Filters.

Klik kanan pada area trace → Properties → tab Events Selection → Column Filters.

Langkah 2 Tambahkan filter Duration.

Field Filter	Nilai
Column	Duration
Operator	Greater than or equal
Value	1000000 (= 1 detik, dalam microseconds)

Satuan Duration di SQL Server Profiler

Duration di SQL Server Profiler diukur dalam MICROSECONDS (bukan milliseconds).

1 detik = 1.000.000 microseconds

500 milidetik = 500.000 microseconds

3 detik = 3.000.000 microseconds

Jadi filter Duration > 1000000 berarti: tampilkan query yang berjalan > 1 detik.

Langkah 3 Klik Run, lalu jalankan query simulasi slow query berikut di SSMS.

```
-- Simulasi slow query: delay 3 detik kemudian jalankan SELECT
WAITFOR DELAY '00:00:03';

SELECT *
FROM aktivitas.Log_Bimbingan;
GO
```

WAITFOR DELAY memerintahkan SQL Server untuk menunggu selama waktu yang ditentukan sebelum melanjutkan eksekusi. Ini mensimulasikan query yang lambat tanpa benar-benar membebani database.

Langkah 4 Amati hasil pada Profiler dan catat.

Kolom	Nilai yang Terekam
TextData	WAITFOR DELAY '00:00:03'; SELECT * FROM aktivitas.Log_Bimbingan
Duration	~3000 ms atau ~3000000 microseconds
StartTime	
EndTime	

Query ini muncul di Profiler dengan Duration sekitar 3000 ms (3 detik), membuktikan filter slow query bekerja. Dalam kondisi nyata, query yang muncul di filter ini adalah kandidat utama untuk dianalisis dan dioptimasi — misalnya dengan menambahkan indeks pada kolom yang digunakan di WHERE.

Rangkuman Praktikum

Materi	Praktikum	Yang Dikerjakan	Hasil yang Diharapkan
10 – SQL Server Agent	P1	Aktifkan SQL Server Agent	Status: Running
10 – SQL Server Agent	P2	Buat Job Backup_Harian_MonitoringTA	Job berhasil dijalankan, file .bak terbuat
10 – SQL Server Agent	P3	Buat Schedule Backup_Malam (23:00)	Backup otomatis setiap malam
10 – SQL Server Agent	P4	Buat Job Rebuild_Index_MonitoringTA	Indeks di-rebuild setiap minggu
10 – SQL Server Agent	P5	Buat Operator Admin_MonitoringTA	Siap menerima notifikasi email
10 – SQL Server Agent	P6	Buat Alert_Backup_Gagal (error 3013)	Email terkirim saat backup gagal
11 – Indeks & Optimasi	P1	Amati query tanpa indeks	Catat Logical Reads baseline
11 – Indeks & Optimasi	P2	Identifikasi Clustered Index	PK_Log_Bimbingan pada ID_Log
11 – Indeks & Optimasi	P3	Buat IDX_Log_Bimbingan_Nim	Logical Reads berkurang, Index Seek
11 – Indeks & Optimasi	P4	Analisis Execution Plan	Pahami Scan vs Seek
11 – Indeks & Optimasi	P5	Buat IDX_Log_Bimbingan_Tanggal	Row Size turun, Cost turun
13 – Monitoring	P1	Activity Monitor real-time	Catat CPU/IO/Waiting Tasks
13 – Monitoring	P2	Recent Expensive Queries	Identifikasi query termahal
13 – Monitoring	P3	PerfMon (CPU, Memory, Disk)	Grafik performa saat beban tinggi
13 – Monitoring	P4	SQL Server Profiler	Rekam query dan durasinya
13 – Monitoring	P5	Filter slow query (> 1 detik)	Deteksi WAITFOR DELAY di Profiler